

HR System Project Documentation

This document describes the current `hr\_system` Python codebase in depth. Each Python file is explained with its role, flow, and key code behavior.

Overview

The project implements an HR automation backend with a FastAPI service, an LLM-backed classifier, semantic similarity fallback, memory management, audit logging, and a Flask web frontend.

It routes user requests into dedicated HR agents for scheduling, leave management, compliance guidance, or clarification. Persistence includes short-term memory in memory manager and long-term memory in SQLite.

`main.py`

This entry-point file is empty. It exists to satisfy a Python project root and may be used for future script-level initialization.

Purpose

Key code segments

`run.py`

Starts the FastAPI application using Uvicorn with auto-reload enabled for local development.

Purpose

This module is the application launcher for development. It instructs Uvicorn to run the FastAPI app defined in `app.main`.

Key code segments

```
import uvicorn

if __name__ == "__main__":
    uvicorn.run(
        "app.main:app",
        host="0.0.0.0",
        port=8000,
        reload=True
    )
```

`flask\_app.py`

Provides the Flask web frontend that proxies UI interactions to the FastAPI backend endpoints.

Purpose

Key code segments

```
import httpx
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)
app.config["SECRET_KEY"] = "replace-with-your-secret"

TEMPLATE_NAME = "index.html"

def fetch_json(endpoint: str, payload=None):
    try:
        if payload is None:
            response = httpx.get(endpoint, timeout=30.0)
        else:
            response = httpx.post(endpoint, json=payload, timeout=30.0)

        response.raise_for_status()
        return response.json(), None
    except Exception as exc:
        return None, str(exc)

def _get_request_data():
    data = request.get_json(silent=True)
    if data is None:
        data = request.form.to_dict() or {}
    return data

@app.route("/", methods=["GET"])
def index():
    return render_template(
        TEMPLATE_NAME,
        backend_url="http://localhost:8000",
        user_id="user-123",
        message="Please schedule a meeting with my manager.",
    )

@app.route("/submit", methods=["POST"])
def submit_request():
    data = _get_request_data()
    backend_url = data.get("backend_url", "http://localhost:8000").rstrip("/")
    user_id = data.get("user_id", "user-123")
    message_text = data.get("message", "")

    request_result, error_message = fetch_json(
        f"{backend_url}/request",
        payload={"user_id": user_id, "message": message_text},
    )
    return jsonify({"result": request_result, "error": error_message})

@app.route("/audit", methods=["POST"])
def load_audit():
```

```

data = _get_request_data()
backend_url = data.get("backend_url", "http://localhost:8000").rstrip("/")

audit_result, error_message = fetch_json(f"{backend_url}/audit")
return jsonify({"result": audit_result, "error": error_message})

@app.route("/memory", methods=["POST"])
def load_memory():
    data = _get_request_data()
    backend_url = data.get("backend_url", "http://localhost:8000").rstrip("/")
    user_id = data.get("user_id", "user-123")

    memory_result, error_message = fetch_json(f"{backend_url}/memory/{user_id}")
    return jsonify({"result": memory_result, "error": error_message})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8500, debug=True)

```

`streamlit\_app.py`

Includes a Streamlit UI alternative for the backend, although the current user request is to use Flask.

Purpose

Key code segments

```

import json

import httpx
import streamlit as st

st.set_page_config(page_title="HR Multi-Agent UI", layout="wide")

st.title("HR Multi-Agent Dashboard")

api_base = st.text_input("FastAPI base URL", "http://localhost:8000")
user_id = st.text_input("User ID", "user-123")
message = st.text_area("Message", "Please schedule a meeting with my manager.")

col1, col2, col3 = st.columns(3)

with col1:
    if st.button("Submit Request"):
        if not user_id or not message:
            st.warning("Please enter both a User ID and a Message.")
        else:
            endpoint = f"{api_base.rstrip('/')}/request"
            try:
                response = httpx.post(
                    endpoint,
                    json={"user_id": user_id, "message": message},
                    timeout=30,
                )
                response.raise_for_status()
                st.success("Request submitted successfully.")
                st.json(response.json())
            except Exception as exc:

```

```

        st.error(f"Unable to send request: {exc}")

with col2:
    if st.button("Load Audit Log"):
        endpoint = f"{api_base.rstrip('/')}/audit"
        try:
            response = httpx.get(endpoint, timeout=30)
            response.raise_for_status()
            st.json(response.json())
        except Exception as exc:
            st.error(f"Unable to load audit log: {exc}")

with col3:
    if st.button("Load Memory"):
        if not user_id:
            st.warning("Please enter a User ID to load memory.")
        else:
            endpoint = f"{api_base.rstrip('/')}/memory/{user_id}"
            try:
                response = httpx.get(endpoint, timeout=30)
                response.raise_for_status()
                st.json(response.json())
            except Exception as exc:
                st.error(f"Unable to load memory: {exc}")

st.markdown("---")

with st.expander("Current Configuration"):
    st.write(
        {
            "FastAPI base URL": api_base,
            "User ID": user_id,
            "Message": message,
        }
    )

```

`app/config.py`

Loads application settings from `.env`. Configures API keys, OpenAI model names, database path, timeouts, and retries.

Purpose

Key code segments

```

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    APP_NAME: str
    APP_ENV: str
    DEBUG: bool

    OPENAI_API_KEY: str
    OPENAI_BASE_URL: str = "https://api.openai.com/v1"
    MODEL_NAME: str
    EMBEDDING_MODEL_NAME: str = "text-embedding-3-small"

    SQLITE_DB: str

    REQUEST_TIMEOUT: int = 30

```

```

MAX_RETRIES: int = 3

class Config:
    env_file = ".env"

settings = Settings()

```

`app/database.py`

Defines the SQLAlchemy engine, session factory, and declarative base for SQLite persistence.

Purpose

Key code segments

```

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

from app.config import settings

DATABASE_URL = f"sqlite:/// {settings.SQLITE_DB}"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

`app/exceptions.py`

Declares custom exception classes used for classification, agent execution, and memory handling.

Purpose

Key code segments

```

class ClassificationError(Exception):
    pass

class AgentExecutionError(Exception):
    pass

class MemoryError(Exception):
    pass

```

`app/main.py`

Creates the FastAPI app, applies global exception handling, and mounts API routers.

Purpose

Key code segments

```

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

from app.config import settings
from app.database import Base, engine
from app.api.routes import router
from app.api.health import health_router

Base.metadata.create_all(bind=engine)

app = FastAPI(title=settings.APP_NAME)

@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    if isinstance(exc, HTTPException):
        return JSONResponse(
            status_code=exc.status_code,
            content={"detail": exc.detail}
        )

    return JSONResponse(
        status_code=500,
        content={
            "status": "error",
            "message": "Internal server error. Please try again later."
        }
    )

app.include_router(router)
app.include_router(health_router)

```

`app/schemas.py`

Defines request and response Pydantic models for request payloads, agent responses, audit records, and memory records.

Purpose

Key code segments

```
from typing import Optional, List
from pydantic import BaseModel
```

```
class RequestPayload(BaseModel):
    user_id: str
    message: str
```

```
class AgentResponse(BaseModel):
    request_id: str
    intent: str
    confidence: float
    response: str
    agent_used: str
    context_used: Optional[dict]
```

```
class AuditRecord(BaseModel):
    request_id: str
    user_id: str
    intent: str
    confidence: float
    agent: str
    status: str
    created_at: Optional[str]
```

```
class Config:
    from_attributes = True
```

```
class MemoryRecord(BaseModel):
    user_id: str
    content: str
    significance: float
    created_at: Optional[str]
```

```
class Config:
    from_attributes = True
```

`app/api/health.py`

Exposes a simple health check endpoint returning `status: healthy`.

Purpose

Key code segments

```
from fastapi import APIRouter
```

```

health_router = APIRouter()

@health_router.get("/health")
def health_check():
    return {
        "status": "healthy"
    }

```

`app/api/routes.py`

Contains the main FastAPI routes to process requests, read the audit log, retrieve memory, and clear memory.

Purpose

Key code segments

```

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import HTMLResponse
from sqlalchemy.orm import Session

from app.database import get_db
from app.schemas import AgentResponse, RequestPayload
from app.agents.orchestrator import Orchestrator
from app.audit.models import AuditLog
from app.memory.ltm import LongTermMemoryRecord

router = APIRouter()

orchestrator = Orchestrator()

@router.post("/request", response_model=AgentResponse)
def process_request(payload: RequestPayload, db: Session = Depends(get_db)):
    try:
        return orchestrator.process(
            db,
            payload.user_id,
            payload.message
        )
    except Exception:
        raise HTTPException(
            status_code=500,
            detail="Unable to process request right now. Please try again later."
        )

@router.get("/audit")
def get_audits(db: Session = Depends(get_db)):
    logs = db.query(AuditLog).order_by(AuditLog.created_at.desc()).all()
    return [
        {
            "request_id": log.request_id,
            "user_id": log.user_id,
            "intent": log.intent,
            "confidence": log.confidence,
            "agent": log.agent,
            "status": log.status,
            "created_at": log.created_at.isoformat() if log.created_at else None,
        }
    ]

```



```

        for log in logs
    ]

@router.get("/memory/{user_id}")
def get_memory(user_id: str, db: Session = Depends(get_db)):
    # Retrieve STM from the volatile storage
    stm_content = orchestrator.memory_manager.stm.get(user_id)

    records = db.query(LongTermMemoryRecord).filter(
        LongTermMemoryRecord.user_id == user_id
    ).order_by(LongTermMemoryRecord.created_at.desc()).all()

    return {
        "user_id": user_id,
        "short_term_memory": stm_content,
        "long_term_memory": [
            {
                "content": record.content,
                "significance": record.significance,
                "created_at": record.created_at.isoformat() if record.created_at else None,
            }
            for record in records
        ]
    }

@router.delete("/memory/{user_id}")
def clear_memory(user_id: str, db: Session = Depends(get_db)):
    records = db.query(LongTermMemoryRecord).filter(
        LongTermMemoryRecord.user_id == user_id
    )

    deleted = records.delete()
    db.commit()

```

`app/audit/logger.py`

Implements audit logging by creating records with UUID request IDs and committing them to the database.

Purpose

Key code segments

```

import uuid

from sqlalchemy.orm import Session

from app.audit.models import AuditLog

class AuditLogger:

    @staticmethod
    def append_log(
        db: Session,
        user_id: str,
        intent: str,
        confidence: float,
        agent: str,
        status: str
    )

```

```

) -> str:

    request_id = str(uuid.uuid4())

    record = AuditLog(
        request_id=request_id,
        user_id=user_id,
        intent=intent,
        confidence=confidence,
        agent=agent,
        status=status
    )

    db.add(record)
    db.commit()

    return request_id

```

`app/audit/models.py`

Defines the SQLAlchemy `AuditLog` model for persisting request audits.

Purpose

Key code segments

```

from sqlalchemy import Column, Integer, String, Float, DateTime
from datetime import datetime

from app.database import Base

class AuditLog(Base):
    __tablename__ = "audit_logs"

    id = Column(Integer, primary_key=True, index=True)
    request_id = Column(String, nullable=False)
    user_id = Column(String, nullable=False)
    intent = Column(String, nullable=False)
    confidence = Column(Float, nullable=False)
    agent = Column(String, nullable=False)
    status = Column(String, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

```

`app/memory/lrm.py`

Defines the long-term memory SQLAlchemy model and persistence helper methods.

Purpose

Key code segments

```

from datetime import datetime

from sqlalchemy import Column, Integer, String, Float, DateTime
from sqlalchemy.orm import Session

from app.database import Base

class LongTermMemoryRecord(Base):
    __tablename__ = "long_term_memory"

    id = Column(Integer, primary_key=True)
    user_id = Column(String)
    content = Column(String)
    significance = Column(Float)
    created_at = Column(DateTime, default=datetime.utcnow)

class LongTermMemory:

    @staticmethod
    def add(
        db: Session,
        user_id: str,
        content: str,
        significance: float
    ):

        record = LongTermMemoryRecord(
            user_id=user_id,
            content=content,
            significance=significance
        )

        db.add(record)
        db.commit()

    @staticmethod
    def retrieve(db: Session, user_id: str):
        return db.query(LongTermMemoryRecord).filter(
            LongTermMemoryRecord.user_id == user_id
        ).order_by(LongTermMemoryRecord.created_at.desc()).all()

```

`app/memory/stm.py`

Implements in-memory short-term memory for recent user messages. It retains up to 5 messages per user.

Purpose

Key code segments

```

from collections import defaultdict
from typing import Dict, List

class ShortTermMemory:

    def __init__(self):
        self.memory: Dict[str, List[str]] = defaultdict(list)
    def add(self, user_id: str, message: str):

```

```

        self.memory[user_id].append(message)

        if len(self.memory[user_id]) > 5:
            self.memory[user_id].pop(0)

    def get(self, user_id: str) -> List[str]:
        return self.memory[user_id]

```

`app/memory/manager.py`

Manages storing messages to short-term memory plus conditional long-term memory storage with significance scoring.

Purpose

Key code segments

```

from sqlalchemy.orm import Session

from app.memory.stm import ShortTermMemory
from app.memory.ltm import LongTermMemory
from app.memory.scorer import SignificanceScorer

class MemoryManager:

    def __init__(self):
        self.stm = ShortTermMemory()

    def store(self, db: Session, user_id: str, message: str):
        self.stm.add(user_id, message)

        significance = SignificanceScorer.score(message)

        if significance >= 0.5:
            LongTermMemory.add(
                db,
                user_id,
                message,
                significance
            )

    def retrieve_context(self, db: Session, user_id: str):
        stm_context = self.stm.get(user_id)
        ltm_context = LongTermMemory.retrieve(db, user_id)

        return {
            "short_term": stm_context,
            "long_term": [x.content for x in ltm_context]
        }

```

`app/memory/scorer.py`

Computes a significance score for text using keyword presence and message length. Scores above 0.5 are stored in long-term memory.

Purpose

Key code segments

```
class SignificanceScorer:

    @staticmethod
    def score(text: str) -> float:
        score = 0.0

        important_keywords = [
            "leave",
            "policy",
            "manager",
            "meeting",
            "schedule",
            "urgent",
            "compliance",
            "salary"
        ]

        for word in important_keywords:
            if word.lower() in text.lower():
                score += 0.2

        score += min(len(text) / 500, 0.5)

        return round(min(score, 1.0), 2)
```

`app/services/llm\_service.py`

Wraps an LLM client from `langchain\_openai` for direct prompt-based invocation.

Purpose

Key code segments

```
from langchain_openai import ChatOpenAI

from app.config import settings

class LLMService:

    def __init__(self):
        self.llm = ChatOpenAI(
            api_key=settings.OPENAI_API_KEY,
            model=settings.MODEL_NAME,
            temperature=0
        )

    def invoke(self, prompt: str):
        response = self.llm.invoke(prompt)
        return response.content
```

`app/services/langgraph\_flow.py`

Orchestrates the request flow using a graph structure. It injects context, classifies intent, and routes to the chosen agent.

Purpose

Key code segments

```
from typing import TypedDict

from langgraph.graph import StateGraph, END

from app.agents.classifier import IntentClassifier
from app.agents.router import AgentRouter


class GraphState(TypedDict):
    query: str
    intent: str
    confidence: float
    response: str
    context: dict
    history: list[str]


classifier = IntentClassifier()
router = AgentRouter()


def inject_context(state: GraphState):
    state["context"] = state.get("context", {})
    state["history"] = state.get("history", [])
    return state


def classify_node(state: GraphState):
    intent, confidence = classifier.classify(state["query"])

    state["intent"] = intent
    state["confidence"] = confidence

    return state


def route_node(state: GraphState):
    agent = router.get_agent(state["intent"])

    result = agent.handle(state["query"], state.get("context", {}))

    state["response"] = result
    state["history"].append(f"User: {state['query']}")
    state["history"].append(f"Assistant: {result}")

    return state


def build_graph():
    workflow = StateGraph(GraphState)

    workflow.add_node("inject_context", inject_context)
    workflow.add_node("classify", classify_node)
    workflow.add_node("route", route_node)
```

```

workflow.set_entry_point("inject_context")

workflow.add_edge("inject_context", "classify")
workflow.add_edge("classify", "route")
workflow.add_edge("route", END)

return workflow.compile()

```

`app/services/openai\_client.py`

Wraps the OpenAI SDK to perform chat completions and embeddings with configured model names and base URL.

Purpose

Key code segments

```

from typing import Dict, List

from openai import OpenAI

from app.config import settings

class OpenAIClient:
    def __init__(self):
        if not settings.OPENAI_API_KEY:
            raise ValueError("OPENAI_API_KEY is required for LLM and embeddings services.")

        base_url = settings.OPENAI_BASE_URL.rstrip("/")
        self.client = OpenAI(
            api_key=settings.OPENAI_API_KEY,
            base_url=base_url
        )
        self.model = settings.MODEL_NAME or "gpt-4o-mini"
        self.embedding_model = settings.EMBEDDING_MODEL_NAME or "text-embedding-3-small"

    def chat(self, messages: List[Dict[str, str]], temperature: float = 0.0, max_tokens: int = 300) -> str:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=messages,
            temperature=temperature,
            max_tokens=max_tokens,
        )

        return response.choices[0].message.content.strip()

    def embed(self, text: str) -> List[float]:
        response = self.client.embeddings.create(
            model=self.embedding_model,
            input=text
        )
        return response.data[0].embedding

```

`app/services/embedding\_service.py`

Provides embeddings using OpenAI and a local fallback when the external embedding call fails. Also computes cosine similarity.

Purpose

Key code segments

```
import re
from typing import List

from app.services.openai_client import OpenAIClient

class EmbeddingService:

    def __init__(self):
        self.client = OpenAIClient()

    def embed(self, text: str) -> List[float]:
        try:
            return self.client.embed(text)
        except Exception:
            return self._local_embed(text)

    @staticmethod
    def _local_embed(text: str) -> List[float]:
        tokens = re.findall(r"\w+", text.lower())
        counts = {}
        for token in tokens:
            if len(token) < 2:
                continue
            counts[token] = counts.get(token, 0) + 1

        vocabulary = [
            "scheduling", "meeting", "leave", "vacation", "compliance",
            "policy", "training", "manager", "salary", "urgent",
            "request", "approve", "cancel", "book", "travel",
            "holiday", "time", "off", "help", "clarify"
        ]

        length = max(sum(counts.values()), 1)
        return [counts.get(word, 0) / length for word in vocabulary]

    @staticmethod
    def cosine_similarity(vector_a: List[float], vector_b: List[float]) -> float:
        dot_product = sum(a * b for a, b in zip(vector_a, vector_b))
        norm_a = sum(a * a for a in vector_a) ** 0.5
        norm_b = sum(b * b for b in vector_b) ** 0.5

        if norm_a == 0 or norm_b == 0:
            return 0.0

        return dot_product / (norm_a * norm_b)
```

`app/agents/classifier.py`

Implements the LLM-led intent classifier with semantic fallback, confidence calibration, and JSON parsing from raw model output.

Purpose

Key code segments

```
import json
import re
from typing import Dict, List, Tuple

from app.services.embedding_service import EmbeddingService
from app.services.openai_client import OpenAIClient

INTENT_DEFINITIONS: Dict[str, str] = {
    "scheduling": (
        "Requests about booking, rescheduling, cancelling, or confirming meetings, interviews, "
        "and calendar events."
    ),
    "leave": (
        "Requests about vacation, sick leave, time off requests, and absence approvals."
    ),
    "compliance": (
        "Requests about company policy, training requirements, regulatory compliance, "
        "and HR rules."
    ),
    "clarification": (
        "Requests that are unclear, out of scope for scheduling, leave, or compliance, "
        "or that need more information before routing."
    )
}

class IntentClassifier:

    def __init__(self):
        self.llm = OpenAIClient()
        self.embedding_service = EmbeddingService()
        self.intent_embeddings: Dict[str, List[float]] = {}

    def _ensure_intent_embeddings(self) -> None:
        if self.intent_embeddings:
            return

        for intent, description in INTENT_DEFINITIONS.items():
            self.intent_embeddings[intent] = self.embedding_service.embed(description)

    def classify(self, text: str) -> Tuple[str, float]:
        clean_text = text.strip()
        llm_intent, llm_confidence = self._llm_classification(clean_text)
        semantic_intent, semantic_similarity = self._semantic_match(clean_text)

        if llm_intent in INTENT_DEFINITIONS and llm_confidence >= 0.35:
            intent = llm_intent
        else:
            intent = semantic_intent

        if llm_intent in INTENT_DEFINITIONS and llm_confidence < 0.4 and semantic_similarity >= 0.7:
            intent = semantic_intent

        confidence = self._calibrate_confidence(llm_confidence, semantic_similarity, intent)
        return intent, confidence

    def _llm_classification(self, text: str) -> Tuple[str, float]:
        prompt = self._build_prompt(text)
        try:
            raw_response = self.llm.chat([
                {"role": "system", "content": "You are an HR request classifier."},
                {"role": "user", "content": prompt}
            ])
```

```

        ], temperature=0.0)
    except Exception:
        return self._semantic_fallback(text)

    parsed = self._parse_response(raw_response)
    if parsed:
        return parsed["intent"], parsed["confidence"]

    return self._semantic_fallback(text)

def _build_prompt(self, text: str) -> str:
    intent_lines = "\n".join(
        f"- {intent}: {description}"
        for intent, description in INTENT_DEFINITIONS.items()
    )
    return (

```

`app/agents/router.py`

Routes classified intents to concrete agent instances. Defaults to ClarificationAgent for unknown intents.

Purpose

Key code segments

```

from app.agents.scheduling_agent import SchedulingAgent
from app.agents.leave_agent import LeaveAgent
from app.agents.compliance_agent import ComplianceAgent
from app.agents.clarification_agent import ClarificationAgent

class AgentRouter:

    def __init__(self):
        self.agents = {
            "scheduling": SchedulingAgent(),
            "leave": LeaveAgent(),
            "compliance": ComplianceAgent(),
            "clarification": ClarificationAgent()
        }

    def get_agent(self, intent: str):
        return self.agents.get(intent, ClarificationAgent())

```

`app/agents/orchestrator.py`

Coordinates the graph, memory manager, and audit logger to handle a request end-to-end.

Purpose

Key code segments

```

from sqlalchemy.orm import Session

from app.memory.manager import MemoryManager
from app.audit.logger import AuditLogger
from app.services.langgraph_flow import build_graph

class Orchestrator:

    def __init__(self):
        self.graph = build_graph()
        self.memory_manager = MemoryManager()

    def process(
        self,
        db: Session,
        user_id: str,
        message: str
    ):

        context = self.memory_manager.retrieve_context(db, user_id)

        initial_state = {
            "query": message,
            "intent": "",
            "confidence": 0.0,
            "response": "",
            "context": context,
            "history": context.get("short_term", [])
        }

        try:
            result = self.graph.invoke(initial_state)
            self.memory_manager.store(db, user_id, message)

            request_id = AuditLogger.append_log(
                db=db,
                user_id=user_id,
                intent=result["intent"],
                confidence=result["confidence"],
                agent=result["intent"],
                status="success"
            )

            return {
                "request_id": request_id,
                "intent": result["intent"],
                "confidence": result["confidence"],
                "response": result["response"],
                "agent_used": result["intent"],
                "context_used": context
            }

        except Exception:
            request_id = AuditLogger.append_log(
                db=db,
                user_id=user_id,
                intent="clarification",
                confidence=0.0,
                agent="clarification",
                status="failed"
            )

            return {
                "request_id": request_id,
                "intent": "clarification",
                "confidence": 0.0,
                "response": "Unable to process your request at this time. Please try again in a few minutes.",
                "agent_used": "clarification",

```

```
        "context_used": context
    }
```

`app/agents/scheduling\_agent.py`

Implements a simple agent response for the specific HR intent.

Purpose

Key code segments

```
from app.agents.base_agent import BaseAgent

class SchedulingAgent(BaseAgent):

    def handle(self, query: str, context: dict) -> str:
        return (
            "Scheduling Agent: Meeting has been tentatively scheduled "
        )
```

`app/agents/leave\_agent.py`

Implements a simple agent response for the specific HR intent.

Purpose

Key code segments

```
from app.agents.base_agent import BaseAgent

class LeaveAgent(BaseAgent):

    def handle(self, query: str, context: dict) -> str:
        return (
            "Leave Agent: Your leave request has been submitted "
            "for manager approval."
        )
```

`app/agents/compliance\_agent.py`

Implements a simple agent response for the specific HR intent.

Purpose

Key code segments

```
from app.agents.base_agent import BaseAgent

class ComplianceAgent(BaseAgent):

    def handle(self, query: str, context: dict) -> str:
        return (
            "Compliance Agent: According to company policy, "
            "all employees must complete compliance training annually."
        )
```

`app/agents/clarification\_agent.py`

Implements a simple agent response for the specific HR intent.

Purpose

Key code segments

```
from app.agents.base_agent import BaseAgent

class ClarificationAgent(BaseAgent):

    def handle(self, query: str, context: dict) -> str:
        return (
            "Clarification Agent: I need more details to process "
            "your request. Could you clarify your intent?"
        )
```

`app/agents/base\_agent.py`

Implements a simple agent response for the specific HR intent.

Purpose

Key code segments

```
from abc import ABC, abstractmethod

class BaseAgent(ABC):

    @abstractmethod
```

```
def handle(self, query: str, context: dict) -> str:  
    pass
```